Mini Game Hub: Project Report

Daksh Maahor (25B0974) and Rujul Garg (25B0937)

April 29, 2026

Abstract

This report summarizes the design and implementation of Mini Game Hub, a two-player game launcher built with Python, Pygame, and shell scripting. It includes installation and running instructions, an overview of code structure, design rationale, and references used during development.

1 Project Design

Mini Game Hub is designed as a modular two-player board game environment with three main responsibilities:

- **Authentication and setup**: a Bash launcher that authenticates users and prepares the Python runtime.
- **Game orchestration**: a Pygame-based GUI that allows players to select games and interact with the board.
- **Game logic**: reusable game rules implemented in Python classes, one per game mode.

The architecture separates low-level game rules from presentation, making it easier to add new games and adapt the UI without rewriting the rule engine.

2 Installation and Running Instructions

The repository provides a dependency declaration file and a launcher script for ease of setup.

2.1 Dependencies

Required Python packages are listed in `requirements.txt`:

- `numpy==2.4.4`
- `pygame==2.6.1`
- `matplotlib==3.9.2`

The launcher script `main.sh` creates a Python virtual environment and installs any missing dependencies automatically.

2.2 Run the Project

From the repository root, execute:

```
bash main.sh
```

This script also:

- creates the persistent data directory `data/`
- initializes `data/users.tsv` and `data/history.csv`
- prompts both players to log in or register
- launches the game GUI using `src/game.py`

3 Code Structure

The project files are organized to keep UI, game logic, utilities, and scripts separate.

3.1 Top-level files

- `main.sh`: Launcher and authentication script. Sets up the virtual environment, installs dependencies, and authenticates two players.
- `leaderboard.sh`: Command-line script for generating a player leaderboard from history data.
- `requirements.txt`: Lists Python package dependencies.
- `README.md`: Project overview, setup hints, and usage notes.

3.2 Source modules

- `src/game.py`: Main Pygame entry point. It renders the hub menu, navigates between screens, and records game results. It also launches the chart dashboard and leaderboard script after a match.
- `src/chart.py`: Generates statistics charts from `data/history.csv` using Matplotlib, showing player win counts, game popularity, and win rates.
- `src/utils/Settings.py`: Stores shared constants for screen dimensions, colors, and UI styling.
- `src/UI/buttons.py`: Shared button rendering helper used across game screens.

3.3 Game logic modules

- `src/games/BoardGame.py`: Base class for all board games. It provides the board state, turn switching, and the contract for the `check_win()` method.
- `src/games/connect4.py`: Connect Four implementation. It manages piece drops, board rendering, and four-in-a-row win detection.
- `src/games/othello.py`: Othello/Reversi implementation. It handles move validation, disc flips, valid-move highlighting, and endgame scoring.
- `src/games/tictactoe.py`: Extended Tic-Tac-Toe on a 10x10 board. It requires five in a row to win and uses sliding-window logic for efficient win detection.

4 System Workflow and Operation

This section describes how Mini Game Hub operates from initialization to displaying post-game statistics.

4.1 Initialization and Authentication

When a user executes `bash main.sh` from the repository root, the following sequence occurs:

1. The `main.sh` script checks for the existence of the `venv/` directory. If it does not exist, a Python virtual environment is created using `python3 -m venv venv`.
2. The script compares installed packages in the virtual environment against `requirements.txt`. Any missing packages are installed via `pip`.
3. The script verifies that the persistent data directory `data/` exists. If not, it is created alongside placeholder files `users.tsv` (with column headers) and `history.csv` (empty or with header row).
4. The script then enters an interactive authentication loop, prompting Player 1 to either log in with an existing username/password or register a new account.
5. Credentials are validated by hashing the input password with SHA-256 and comparing it against the stored hash in `data/users.tsv`.
6. Once Player 1 is authenticated, the script prompts Player 2 similarly, ensuring that the same user cannot log in twice for a single session.
7. After both players authenticate successfully, the script executes `python3 src/game.py <player1_username> <player2_username>`.

4.2 Main Menu and Game Selection

Upon execution, `src/game.py` initializes the Pygame window using constants defined in `src/utils/Settings.py`. The application displays a fade-in welcome screen featuring "Welcome to MiniGameHub", which persists until the user presses any key or clicks the mouse. After the welcome screen, the main menu appears with the following options:

- **Play Connect Four**: Launches a new Connect Four game.
- **Play Othello**: Launches a new Othello game.
- **Play Tic-Tac-Toe**: Launches an extended Tic-Tac-Toe game (10x10 grid with five-in-a-row win condition).
- **View Leaderboard**: Displays cumulative player statistics by invoking the `leaderboard.sh` script in a subprocess.
- **View Help**: Shows an on-screen help card with game descriptions and controls.
- **Exit**: Closes the application.

The menu is rendered using Pygame surfaces and font objects, with mouse click event detection determining which option the player selected.

4.3 Game Execution Flow

When a player selects a game, the corresponding game class is instantiated (e.g., `Connect4`, `Othello`, or `TicTacToe`). Each game class extends `BoardGame`, inheriting turn management and player name tracking. The game loop then:

1. Initializes the board representation (typically a NumPy array of dimensions rows $\times$ cols).
2. Renders the board to the Pygame window, including pieces already placed and any highlighting (e.g., valid moves in Othello, hover previews in Tic-Tac-Toe).
3. Displays the current player's username and turn indicator.
4. Waits for a mouse click or keyboard event from the active player.
5. Translates the pixel coordinates of the click to board coordinates and attempts to execute the move.
6. Validates the move according to the game's rules (e.g., checking for valid captures in Othello, preventing overlapping moves in Tic-Tac-Toe).
7. If valid, updates the board state, calling any game-specific logic (e.g., disc flips in Othello).
8. Switches the active player to the other player.
9. Checks for a win condition or draw by invoking `check_win()`.
10. If the game continues, returns to the rendering step.

4.3.1 Game-Specific Mechanics

- **Connect Four:** The board is 7 rows × 7 columns. Pieces are dropped into columns and fall to the lowest available row. The game is won when a player places four consecutive pieces horizontally, vertically, or diagonally. If all cells are filled with no winner, the game ends in a draw.
- **Othello:** The board is 8×8 with standard Othello starting position (two black and two white discs in the center). A move is valid only if it captures at least one opponent disc with the newly placed disc and a continuous line of opponent discs. Discs are flipped when captured. If no valid moves exist for the current player, the turn passes to the opponent. The game ends when neither player can move; the player with more discs wins, or a draw occurs if counts are equal.
- **Tic-Tac-Toe:** The board is 10×10. Players place either "X" or "O" marks. A player wins by achieving five consecutive marks (horizontally, vertically, or diagonally). The game uses NumPy's sliding-window view to efficiently detect winning sequences. The game ends in a draw if all cells are filled without a winner.

4.4 Result Recording and Post-Game Display

Upon game completion, the following sequence occurs:

1. The game result (winner or "Draw") is formatted and appended to `data/history.csv` as a new row with: timestamp, player 1 name, player 2 name, game type, and result.
2. The application displays a result screen showing the winner/draw message and any final board state.
3. The application automatically invokes `leaderboard.sh` as a subprocess to display an up-to-date terminal leaderboard showing cumulative wins, losses, and draw counts.
4. Next, the application invokes `src/chart.py` to generate Matplotlib figures from `data/history.csv`:
   - A bar chart showing each player's total wins.
   - A pie or bar chart showing the popularity of each game mode.
   - A line chart showing each player's win rate over time.
5. The charts are displayed in a Matplotlib window for visual inspection.
6. The user is prompted to return to the main menu or exit the application.

4.5 Data Persistence

All authentication and gameplay data is persisted to CSV files in the `data/` directory:

- `data/users.tsv`: Tab-separated file with columns: `username`, `password_hash`. Passwords are hashed with SHA-256 to prevent plaintext storage.

- `data/history.csv`: Comma-separated file with columns: `timestamp`, `player1`, `player2`, `game_type`, `result_winner`. Each row represents one completed game.

The `leaderboard.sh` script parses `history.csv` to compute aggregate statistics (wins, losses, draws) for each player, while `src/chart.py` uses the same file to generate visualizations.

5 Detailed Module Responsibilities

5.1 `main.sh`

This script is the user-visible launcher. It:

- creates or verifies `data/users.tsv` and `data/history.csv`
- validates usernames and passwords
- uses SHA-256 hashing for credential storage
- prevents a single user from playing both sides in the same session
- invokes `python3 src/game.py` once two players are authenticated

5.2 `src/game.py`

- renders the welcome screen and main menu
- provides navigation between game modes, leaderboard, and help cards
- records match results to `data/history.csv`
- calls `src/chart.py` and `leaderboard.sh` after matches

5.3 Game classes

Each game class extends `BoardGame` and implements its own board rendering, click handling, and win conditions.

- `Connect4`: stores a 7x7 board and checks horizontal, vertical, and diagonal winning sequences.
- `Othello`: initializes the standard starting layout, computes valid captures in eight directions, and flips opponent discs.
- `TicTacToe`: draws a 10x10 grid, highlights hover previews, and uses NumPy sliding-window views to detect five consecutive marks.

6 Major Functions and API

This section documents the key functions exposed by each module, their parameters, and responsibilities.

6.1 `src/game.py` — Main Application Entry Point

- `main()`: Entry point that initializes Pygame, loads the background image, displays the welcome intro, and enters the main menu loop. Parses command-line arguments for player usernames.
- `game_main(player1, player2)`: Core application orchestrator. Initializes the Pygame window, manages the game loop for the main menu, and coordinates transitions between game selection, gameplay, result display, and statistics.
- `show_intro(screen, bg)`: Displays the fade-in welcome splash screen with the title "Welcome to MiniGameHub". Polls for keyboard or mouse input to proceed.
- `show_menu(screen, font, bg)`: Renders the main menu with six interactive cards (Connect Four, Othello, Tic-Tac-Toe, Leaderboard, How to Play, Exit). Returns the selected action key and handles card hover effects, shadows, and custom icon drawing.
- `show_how_to_play(screen, bg)`: Displays the help card UI with game descriptions, rules, and controls. Renders mini board visualizations for each game mode and provides navigation back to the main menu.
- `show_result(screen, font, winner)`: Shows the game result (winner or draw) and displays the final board state. Prompts the user to continue or exit.
- `run_game(screen, game, bg, game_name)`: Main game loop for active gameplay. Handles event polling, calls the game's `draw_screen()` and `handle_click()`, and monitors for win/draw conditions. Calls `check_win()` repeatedly until the game terminates.
- `show_leaderboard_menu(screen, font, bg)`: Invokes the `leaderboard.sh` script as a subprocess to display terminal-based player statistics.
- `show_message(screen, font, text, duration)`: Displays a temporary status message (default 2 seconds) for feedback like "Invalid move" or connection status.
- `draw_button(screen, text, font, rect, base_color, hover_color, events)`: Utility that renders a clickable button with shadow, hover effect, and border highlight. Returns `True` when clicked.
- `_clean_player_name(name)`: Removes non-printable and non-ASCII characters from usernames to prevent display corruption.
- `_wrap_text(text, font, max_width)`: Splits text into multiple lines to fit within a pixel width, used for multi-line help text and descriptions.
- `_draw_mini_board_connect4(), _draw_mini_board_othello(), _draw_mini_board_tictact`
  Helper functions that render small, decorative board previews within help or menu cards. Used for visual presentation without full gameplay.

6.2 `src/games/BoardGame.py` — Base Game Class

- `__init__(player1, player2, rows, cols)`: Initializes shared board state, player list, and common attributes like `current_turn`, `last_move`, and `win_cells`.
- `switch_turn()`: Toggles `current_turn` between 0 (player 1) and 1 (player 2).
- `get_current_player()`: Returns the username of the active player.
- `check_win()`: Abstract method (contract). Each game subclass implements rule-specific win detection. Typically returns a player name, "Draw", or `None` (game ongoing).

6.3 `src/games/connect4.py` — Connect Four Game

- `handle_click(x, y)`: Translates pixel coordinates to column index. Finds the lowest empty row in that column and places a piece (player 1 or 2). Switches the active player.
- `draw_screen(screen, events)`: Renders the 7×6 board grid with colored pieces, a hover preview showing where a piece will land, and a pulsing ring around the last move for visual feedback.
- `check_win()`: Scans the board for four consecutive pieces horizontally, vertically, or diagonally. Returns the winner, "Draw", or `None`.

6.4 `src/games/othello.py` — Othello/Reversi Game

- `__init__(p1, p2)`: Initializes the 8×8 board with the standard Othello starting position: two black and two white discs in the center.
- `_shift(mask, dr, dc)`: Vectorized NumPy operation that shifts a boolean mask in a direction (e.g., up-left, down, right-down) by padding with zeros. Enables efficient scanning of board patterns.
- `get_valid_moves_mask(player)`: Uses NumPy array operations and the `_shift()` helper to compute all legal moves for a player in a single pass. Returns a 8×8 boolean mask.
- `is_valid_move(row, col, player)`: Scalar check for a single cell. Returns `True` if placing the player's disc at (row, col) captures at least one opponent disc.
- `place_and_flip(row, col, player)`: Places a disc and flips all opponent discs in lines bounded by the placed disc and another friendly disc. Updates the board in place.
- `has_valid_moves(player)`: Returns `True` if the player has at least one legal move. Used to skip turns when a player is blocked.
- `handle_click(x, y)`: Validates the clicked cell, calls `place_and_flip()`, and handles turn passing if the next player has no moves.

- `draw_screen(screen, events)`: Renders the 8×8 grid with black and white discs, highlights valid moves for the current player, and displays a pass button when necessary.
- `check_win()`: Counts discs for each player. Returns the player with more discs, "Draw" if equal, or `None` if the game continues.

6.5 `src/games/tictactoe.py` — Extended Tic-Tac-Toe Game

- `__init__(p1, p2)`: Initializes the 10×10 board with no starting pieces (empty grid).
- `_get_board_geometry()`: Computes and caches the board's screen offset and dimensions. Ensures consistent coordinate translation between pixel and board space.
- `_get_hover_cell()`: Returns the current ((row, col)) under the mouse, or `None` if outside the board. Used to render hover preview in the next available cell.
- `handle_click(x, y)`: Translates pixel coordinates to (row, col). Places the current player's mark (1 for X, 2 for O) if the cell is empty. Switches the active player.
- `draw_screen(screen, events)`: Renders the 10×10 grid with X and O marks, a hover preview showing where the current player would place, and highlights winning cells if the game has ended.
- `check_win()`: Uses NumPy's `sliding_window_view()` to scan for five consecutive marks horizontally, vertically, or diagonally. Caches winning cell coordinates in `self.win_cells` for highlighting. Returns the winner, "Draw", or `None`.

6.6 `src/UI/buttons.py` — UI Utilities

- `draw_button(screen, text, font, rect, base_color, hover_color, events)`: Renders a button with drop shadow, border highlight (glow on hover), and text. Detects left-mouse click within the button's bounding box and returns `True` on activation. Used for navigation buttons throughout the UI.

6.7 `src/chart.py` — Statistics Visualization

- `Parse history file`: Reads `data/history.csv` and aggregates game results into counters for player wins, game popularity, and per-player game counts.
- `Generate four subplots`: Uses Matplotlib to create a 2×2 figure showing: (1) top 5 players by wins, (2) game popularity (pie chart), (3) top 5 most active players, and (4) win rates for leading players. Displays the figure in an interactive window.

7 References

1. Pygame Documentation. `https://www.pygame.org/docs/`. Accessed April 2026.
2. NumPy User Guide. `https://numpy.org/doc/stable/user/index.html`. Accessed April 2026.

3. Matplotlib Documentation. https://matplotlib.org/stable/contents.html. Accessed April 2026.

4. Python Documentation. https://docs.python.org/3/. Accessed April 2026.

5. Bash Reference Manual. https://www.gnu.org/software/bash/manual/bash.html. Accessed April 2026.